

Project 4

2D Convolution Acceleration with gem5 Custom Instruction and RTL Memory-Mapped Accelerator

Weight: 30 points

Submission format: One ZIP file uploaded to the course submission system

Submission format & Deadline: At 12:00 on 06/11.

Individual work rule

This project must be completed individually. You may discuss high-level course concepts with classmates, but your code, experiments, measurements, analysis, and report must be entirely your own work. The use of ChatGPT or any other large language model (LLM), AI writing assistant, AI coding assistant, or automated paraphrasing/generation tool is strictly prohibited for this assignment. This includes, but is not limited to:

- generating code,
- generating explanations,
- generating report text,
- generating answers to analysis questions,
- rewriting your draft.

Any use of such tools in preparing or submitting this assignment will be treated as academic misconduct.

Project objective

Build and evaluate a workload-specific accelerator path for 2D convolution using both gem5 and SystemVerilog RTL. In gem5, you will model a RISC-V custom instruction that accelerates packed dot-product operations used by a 3x3 convolution kernel. In RTL, you will implement only the memory-mapped convolution accelerator block; a provided RISC-V core, instruction memory, data memory, MMIO decoder, workload program, and testbench will drive and verify your accelerator.

Workload

The workload is a fixed 3x3 2D convolution over a small image. Boundary pixels are skipped, so the kernel operates only on interior pixels. Input pixels are unsigned 8-bit values, kernel coefficients are signed 8-bit values, and each output is a signed 32-bit accumulated result.

For each interior location, the computation is:

$$\text{out}[y][x] = \text{sum over a 3x3 input window of } \text{pixel}[y+dy][x+dx] * \text{kernel}[dy][dx]$$

Required project components

Part	Component	Required work
Part A	gem5 scalar baseline	Run the provided scalar 2D convolution workload and collect baseline statistics.
Part B	gem5 custom instruction	Add and validate a RISC-V custom xdot4 instruction for four packed 8-bit products.
Part C	gem5 optimized workload	Modify the convolution kernel to use xdot4 and compare against scalar output.
Part D	RTL accelerator	Implement only conv_accel.sv, the memory-mapped convolution accelerator.
Part E	Analysis report	Explain correctness, performance, speedup limits, and RTL/gem5 differences.

gem5 custom instruction requirement

Students must implement a custom RISC-V instruction named xdot4. The instruction treats rs1 and rs2 as four packed signed 8-bit values and returns the 32-bit sum of four signed products.

Required semantic behavior:

$$rd = rs1[7:0]*rs2[7:0] + rs1[15:8]*rs2[15:8] + rs1[23:16]*rs2[23:16] + rs1[31:24]*rs2[31:24]$$

Students must document the exact encoding they used, the gem5 files they modified, and how the instruction is invoked from C.

RTL system provided to students

The starter package provides a complete RISC-V-controlled RTL system. Students do not implement the processor, instruction memory, data memory, MMIO decoder, top-level module, workload program, or testbench. These are provided so the project remains focused on accelerator implementation and interface correctness.

File	Status	Purpose
rv32i_core.sv	Provided	Small RV32I subset core that executes the accelerator-control program.
instr_mem.sv	Provided	Loads the supplied program_conv.hex workload.
data_mem.sv	Provided	Stores accelerator outputs for the testbench checker.
mmio_decoder.sv	Provided	Routes normal memory accesses and accelerator MMIO accesses.
project4_top.sv	Provided	Connects the core, memories, decoder, and accelerator.
tb_project4.sv	Provided	Runs the program and checks expected results.
conv_accel.sv	Student TODO	Memory-mapped convolution accelerator to implement.

RTL accelerator register map

Offset	Register	Direction	Meaning
0x00	CTRL	Write	bit 0 = start convolution
0x04	STATUS	Read	bit 0 = done, bit 1 = busy
0x08	RESULT	Read	signed 32-bit convolution result
0x10-0x30	PIXELO-PIXEL8	Write	unsigned 8-bit input window values in wdata[7:0]
0x40-0x60	KERNEL0-KERNEL8	Write	signed 8-bit kernel coefficients in wdata[7:0]

What you must implement

- Memory-mapped register writes for nine pixels and nine kernel coefficients.
- CTRL/start behavior that begins one convolution operation.
- STATUS behavior with correct done and busy bits.
- RESULT behavior returning the signed 32-bit convolution output.
- Correct 3x3 multiply-accumulate datapath using unsigned pixels and signed coefficients.
- Correct behavior across multiple consecutive accelerator invocations from the provided RISC-V program.
- gem5 xdot4 custom instruction implementation and optimized C workload integration.
- Correctness checks showing scalar, optimized, xdot4, and RTL outputs agree.

Required computing environment

All experiments for this project must be run using the following Docker image:

amansinhaatnycu/osp:latest

Pull: `docker pull amansinhaatnycu/osp:latest`

Run example: `docker run --rm -it -v ./work -w /work amansinhaatnycu/osp:latest bash`

Recommended implementation order

- First run `make run_rtl` and observe that the starter accelerator is incomplete.
- Implement the `conv_accel.sv` register map before implementing the datapath.
- Add start, busy, done, and result behavior.
- Implement the 3x3 signed multiply-accumulate computation.
- Confirm the provided RISC-V program halts and the RTL testbench prints PASS.
- Run gem5 scalar and optimized C workloads.
- Add the xdot4 instruction in gem5 and replace the C fallback wrapper.
- Collect and compare gem5 statistics for scalar, optimized, and xdot4 versions.

Deliverables

- Completed gem5 modifications or patch files for the xdot4 instruction.
- Completed `conv2d_xdot4.c` using the custom instruction wrapper.
- Completed `rtl/conv_accel.sv` implementation.
- Verilator RTL log showing the RISC-V-controlled accelerator test passing.
- gem5 output logs and selected statistics for scalar, optimized, and xdot4 versions.
- A short report, maximum 7 pages, explaining implementation, correctness, performance, and limitations.

Report questions

- 1. Why is 2D convolution suitable for acceleration?
- 2. Which operations dominate the scalar implementation?
- 3. How does xdot4 reduce dynamic instruction count?
- 4. Why can speedup be smaller than arithmetic-instruction reduction?
- 5. How do row-major traversal and cache locality affect performance?
- 6. How does the RTL memory-mapped accelerator differ from the gem5 custom instruction?
- 7. What is the hardware/software contract between the RISC-V core and conv_accel.sv?
- 8. What limitations remain in this project compared with a real convolution accelerator?

Grading rubric

Category	Points	What is being evaluated
Scalar 2D convolution baseline in gem5	4	Correct execution, baseline stats, checksum evidence.
gem5 custom instruction implementation	6	Decode, semantics, rebuild, execution, and documentation.
Optimized convolution using xdot4	5	Correct wrapper/use, output match, meaningful workload integration.
gem5 performance analysis	5	simInsts, cycles, CPI/IPC, speedup, and bottleneck explanation.
RTL convolution accelerator implementation	6	MMIO registers, start/done/busy, signed 3x3 MAC, result behavior.
RTL system execution and verification	3	Provided RISC-V program successfully drives accelerator and testbench passes.
Report clarity and reproducibility	1	Clear commands, logs, limitations, and organization.

Student expectations

A strong submission treats gem5 and RTL as two views of the same accelerator idea. The gem5 extension is an architectural performance model for a packed dot-product primitive. The RTL accelerator is a memory-mapped prototype component controlled by a RISC-V program. Your report should compare these levels explicitly rather than presenting them as unrelated tasks.